
UNIT 11 INTRODUCTION TO NOSQL AND BIGDATA

Structure

- 11.0 Introduction
- 11.1 Objectives
- 11.2 Introduction to NoSQL
 - 11.2.1 What is NoSQL
 - 11.2.2 Brief History of NoSQL Databases
 - 11.2.3 NoSQL Database Features
 - 11.2.4 Differentiate between RDBMS and NoSQL
- 11.3 Types of NoSQL Databases
 - 11.3.1 Column Based
 - 11.3.2 Graph Based
 - 11.3.3 Key-value Pair Based
 - 11.3.4 Document Based
- 11.4 Introduction to Big Data
 - 11.4.1 Big Data and Characteristics
 - 11.4.2 Big Data Applications
 - 11.4.3 Structured vs Semi-structured and Unstructured Data
 - 11.4.4 Big Data vs Data Warehouse
 - 11.4.5 Distributed File System
 - 11.4.6 HDFS and Map Reduce
 - 11.4.7 Apache Hadoop 1 and 2 (YARN)
- 11.5 Summary
- 11.6 Solutions/Answers
- 11.7 Further Readings

11.0 INTRODUCTION

NoSQL databases have significantly reshaped the way data is stored and accessed, even though relational databases remain widely utilized. Modern applications often feature advanced functionalities like Google-style search, reflecting a shift in user expectations and data handling needs. The surge in data generation, along with the proliferation of internet usage, mobile technologies, and the rising importance of analytics, has led to new demands on database systems. In response to these evolving requirements, NoSQL databases have emerged and grown rapidly, offering various models such as key-value stores, document databases, column-oriented databases, and graph-based systems.

In the current digital landscape, enormous volumes of data—measured in terabytes—are produced every day from a vast range of sources. These include social media platforms, corporate systems (such as production data, customer records, and financial information), user interactions, sensor data,

and digital devices like smart phones and vehicles. This overwhelming and continuous generation of data is categorized under the umbrella term "Big Data." The concept of Big Data is defined by four key characteristics: volume (the sheer amount of data), velocity (the speed at which data is generated and processed), variety (the different types and formats of data), and veracity (the reliability and accuracy of data).

The discipline that focuses on extracting valuable insights from this vast data landscape is known as **Big Data Analytics**. It requires sophisticated tools and technologies to process and analyze data effectively. The structural design that supports this process is referred to as **Big Data Architecture**. This architecture outlines how data is collected from various sources, processed and stored, analyzed for insights, and ultimately delivered to the end-user or business decision-maker.

At the heart of Big Data Analytics lies the Big Data Architecture, which is typically divided into four logical layers. Each of these layers plays a vital role in managing and analyzing large-scale data:

- **Big Data Source Layer:** This initial layer manages both batch and real-time data input from various origins, including traditional data warehouses, relational database systems, SaaS platforms, and Internet of Things (IoT) devices.
- **Management and Storage Layer:** Data received from the sources is transformed into formats suitable for analytics tools and then stored accordingly in this layer.
- **Analysis Layer:** Here, the stored data is examined to uncover patterns, trends, and business intelligence insights.
- **Consumption Layer:** Finally, this layer ensures that the results of the analysis are made accessible to users or integrated into business intelligence systems for strategic decision-making.

In addition to these layers, Big Data Architecture also supports several essential functions:

- **Connecting to Data Sources:** With the help of connectors and adapters, the system can access a diverse array of storage types, network protocols, and data formats.
- **Data Governance:** This function ensures that privacy and security regulations are applied throughout the data lifecycle—from ingestion to storage and eventual deletion.
- **Systems Management:** As these architectures are generally deployed across large, distributed clusters, they require constant monitoring and control through centralized management tools.
- **Quality of Service Assurance:** This framework helps maintain high standards of data quality, defines compliance policies, and manages the frequency and volume of data intake.

In this Unit, we will delve into the various types of NoSQL databases, particularly those based on key-value pairs, documents, columns, and graphs.

Furthermore, we will explore the concept of Big Data in depth—its key characteristics and the broad spectrum of applications it supports.

11.1 OBJECTIVES

After going through this unit, you should be able to:

- define what is NoSQL;
- differentiate between NoSQL and SQL;
- explain the basic features of Column based NoSQL Database;
- explain the Graph-based NoSQL Database;
- explain the Key-value pair based NoSQL Database and
- explain the Document based NoSQL Database.
- explain Big data and its Characteristics
- compare the characteristics of Structured Semi-Structured, and Unstructured data
- differentiate Big data and Data warehouse
- describe the Distributed file system
- explain HDFS and Map Reduce
- differentiate between Apache Hadoop 1 and 2 (YARN).

11.2 INTRODUCTION TO NoSQL

Databases are a crucial part of many technological and practical systems. The phrase "NoSQL database" is frequently used to describe any non-relational database. NoSQL is sometimes referred to as "non SQL," but it is also referred to as "not only SQL." In either case, the majority of people agree that a NoSQL database is a type of database that stores data in a format that is different from relational tables.

Whenever you want to use the data, it must first be saved in a particular structure and then converted into a usable format. On the other hand, there are some circumstances in which the data are not always presented in a structured style, which means that their schemas are not always rigorous. This unit provides an in-depth look into NoSQL and the features that make it unique.

11.2.1 What is NoSQL?

NoSQL is a way to build databases that can accommodate many different kinds of information, such as key-value pairs, multimedia files, documents, columnar data, graphs, external files, and more. In order to facilitate the development of cutting-edge applications, NoSQL was designed to work with a variety of different data models and schemas.

The great functionality, ease of development, and performance at scale offered by NoSQL have helped make it a popular name. NoSQL is sometimes referred to as a non-relational database due to the numerous data

handling features it offers. Due to the fact that it does not adhere to the guidelines established by Relational Database Management Systems (RDBMS), you cannot query your data using conventional SQL commands. We can think of such well-known examples as Mongo DB, Neo4J, Hyper Graph DB, etc.

11.2.2 Brief history of NoSQL Databases

In the late 2000s, as the price of storage began to plummet, No-SQL databases began to gain popularity. No longer is it necessary to develop a sophisticated, difficult-to-manage data model to prevent data duplication. Because developers' time was quickly surpassing the cost of data storage, NoSQL databases were designed with efficiency in mind.

Table 11.1: History of Databases

Year	Database Solutions	Company / Database Technology
1970-2000	Mainly RDBMS related	Oracle, IBM DB2, SQL Server, MySQL
2000-2005	Dot Com boom – new scale solutions, start of NoSQL dev, whitepapers	Google, Facebook, IBM, amazon
2005-2010	New open source & mainstream databases	Cassandra, Riak, Apache Hbase, neo4j, MongoDB, CouchDB, Redis
2010 onwards	Adoption of Cloud	DBaaS (Database as a Service)

As storage costs reduced significantly, the quantity of data that applications were required to store and query grew. This data came in all forms—structured, semi-structured, and unstructured — and sizes making it practically difficult to define the schema in advance. NoSQL databases give programmers a great deal of freedom by enabling them to store enormous amounts of unstructured data.

In addition, the Agile Manifesto was gaining momentum, and software developers were reconsidering their approach to software development. They were beginning to understand the need of being able to quickly adjust to ever-evolving requirements. They required the flexibility to make rapid iterations and adjustments to all parts of their software stack, including the underlying database. They were able to achieve this flexibility because of NoSQL databases.

The use of the public cloud as a platform for storing and serving up data and applications was another trend that arose, as cloud computing gained popularity. To make their applications more robust, to expand out rather than up, and to strategically position their data across geographies, they needed the option to store data across various servers and locations. These features are offered by some NoSQL databases like MongoDB.

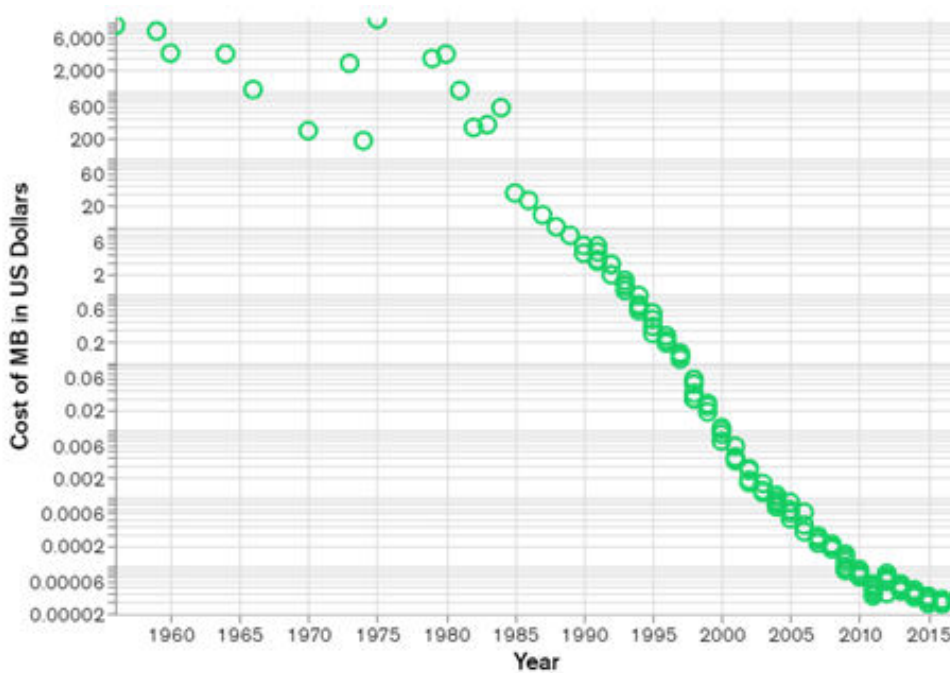


Figure 11.1: Cost per MB of Data over Time (Log Scale)
(Adapted from <https://www.mongodb.com>)

11.2.3 NoSQL Database Features

Every NoSQL database comes with its own set of one-of-a-kind capabilities. The following are general characteristics shared by several NoSQL databases:

- Schema flexibility
- Horizontal scaling
- Quick responses to queries as a result of the data model
- Ease of use for software developers

11.2.4 Difference between RDBMS and NoSQL

The differences and similarities between the two DBMSs are as follows:

For the most part, NoSQL databases fall under the category of non-relational or distributed databases, while SQL databases are classified as Relational Database Management Systems (RDBMS).

Databases that use the Structured Query Language (SQL) are table-oriented, while NoSQL databases use either document-oriented or key-value pairs or wide-column stores, or graph databases.

Unlike NoSQL databases, which have dynamic or flexible schema to manage unstructured data, SQL databases have a strict or static schema.

Structured data is stored using SQL, whereas both structured and unstructured data can be stored using NoSQL.

SQL databases are thought to be scalable in a vertical direction, whereas NoSQL databases are thought to be scalable in a horizontal direction.

Increasing the computing capability of your hardware is the first step in the

scaling process for SQL databases. In contrast, NoSQL databases scale by distributing the load over multiple servers.

MySQL, Oracle, PostgreSQL, and Microsoft SQL Server are all examples of SQL databases. BigTable, MongoDB, Redis, Cassandra, RavenDb, Hbase, CouchDB, and Neo4j are a few examples of NoSQL databases.

Vertical scalability is required for SQL databases. This means that an excessive amount of load must be able to be managed by increasing the amount of CPU, SSD, RAM, GPU, etc. on your server. When it comes to NoSQL databases, the ability to scale horizontally is one of their defining characteristics. This means that the addition of additional servers will make the task of managing demand more manageable.

☞ Check Your Progress 1

- 1) What is NoSQL?

.....
.....
.....

- 2) What are the features of NoSQL databases?

.....
.....
.....

- 3) Differentiate between the NoSQL and SQL.

.....
.....
.....

11.3 TYPES OF NOSQL DATABASES

In this section, we will discuss the many classifications of NoSQL databases. There are typically four types of NoSQL databases:

- 1) Column-based: Instead of accumulating data in rows, this method organizes it all together into columns, which makes it easier to query large datasets.
- 2) Graph-based: These are systems that are utilized for the storage of information regarding networks, such as social relationships.
- 3) Key-value pair based: This is the simplest sort of database, in which each item of your database is saved in the form of an attribute name (also known as a "key") coupled with the value.
- 4) Document-based: Made up of sets of key-value pairs that are kept in documents.

11.3.1 Column Based

A column store, in contrast to a relational database, is arranged as a set of columns, rather than rows. This allows you to read only the columns you need for analysis, saving memory space that would otherwise be taken up by irrelevant information. Because columns are frequently of the same kind, they are able to take advantage of more efficient compression, which makes data reading even quicker. The value of a specific column can be quickly aggregated using columnar databases.

Although columnar databases are excellent for analytics, because of the way they publish data, it is challenging for them to remain consistent because writes to all the columns need several write events on the disk. However, this problem never arises with relational databases because row data is continuously written to disk.

How Does a Column Database Work?

A columnar database is a type of database management system (DBMS) that allows data to be stored in columns rather than rows. It is accountable for reducing the amount of time needed to return a certain query. Additionally, it is accountable for the significant enhancement of the disk I/O performance. Both data analytics and data warehousing benefit from it. Additionally, the primary goal of a Columnar Database is to read and write data in an efficient manner. Column-store databases include Casandra, CosmoDB, Bigtable, and HBase, to name a few.

Columnar Database Vs Row Database:

When processing Big data analytics and data warehousing, there are a number of different techniques that can be used, including columnar databases and row databases. But they each take a different method.

For instance:

- Row Database: "Customer 1: Name, Address, Location". (The fields for each new record are stored in a long row).
- Columnar Database: "Customer 1: Name, Address, Location". (Each field has its own set of columns). Refer Table 2 for relational database example.

Table 11.2: Relational database: an example

ID Number	First Name	Last Name	Amount
A01234	Sima	Kaur	4000
B03249	Tapan	Rao	5000
C02345	Srikant	Peter	1000

In a Columnar DBMS, the data will be stored in the following format:

A01234, B03249, C02345; Sima, Tapan, Srikant; Kaur, Rao, Peter; 4000, 5000, 1000.

In a Row-oriented DBMS, the data will be stored in the following format:

A01234, Sima, Kaur, 4000; B03249, Tapan, Rao, 5000; C02345, Srikant, Peter, 1000.

Columnar databases: advantages

The use of columnar databases has various advantages:

- Column stores are highly effective in compression, making them storage efficient. This implies that you can conserve disk space while storing enormous amounts of data in a single column.
- Aggregation queries are fairly quick with column-store databases because the majority of the data is kept in a column, which is beneficial for projects that need to execute a lot of queries quickly.
- Load times are also quite good; a table with a billion rows can be loaded in a matter of seconds. This suggests that you can load and query practically instantly.
- A great deal of versatility because columns do not have to resemble one another. The database would not be affected if you add new or different columns, however, updating all tables is necessary to input whole new record queries.
- Overall, column-store databases are excellent for analytics and reporting due to their quick query response times and capacity to store massive volumes of data without incurring significant costs.

Column databases: Disadvantages

While there are many benefits to adopting column-oriented databases, there are also a few drawbacks to keep in mind.

- It takes a lot of time and effort to create an efficient indexing schema.
- Incremental data loading is undesirable and is to be avoided, if at all possible, even though this might not be a problem for some users.
- This applies to all forms of NoSQL databases, not just those with columns. Web applications frequently have security flaws, and the absence of security features in NoSQL databases does not help. If security is your top goal, you should either consider using relational databases or, if it's possible, use a clearly specified schema.
- Due to the way data is stored, Online Transaction Processing (OLTP) applications are incompatible with columnar databases.

Are column databases always NoSQL?

Before we conclude, we should note that column-store databases are not always NoSQL-only. It is frequently argued that column-store belongs firmly in the NoSQL camp because it differs so much from relational database approaches. The debate between NoSQL and SQL is generally quite nuanced, therefore this is not usually the case. They are essentially the same as SQL techniques when it comes to column-store databases. For instance, keyspaces

function as schema, so schema management is still necessary. A NoSQL data store's keyspace contains all column families. The concept is comparable to relational database management systems' schema. There is typically only one keyspace per program. Another illustration is the fact that the metadata occasionally resembles a conventional relational DBMS perfectly. Ironically, column-store databases frequently adhere to ACID and SQL standards. However, NoSQL databases are often either document-store or key-store, neither of which are column-store. Therefore, it is difficult to claim that column-store is a pure NoSQL system.

11.3.2 Graph Based

The initial hardware hurdles that made it feasible for SQL to handle vast quantities of data are no longer there, despite the fact that SQL is an excellent superb RDBMS and has been used for many years to manage massive amounts of data. As a result, NoSQL has rapidly emerged as the dominant form of contemporary database management and many of the largest websites we rely on today are powered by NoSQL, like Twitter's use of FlockDB and Amazon's DynamoDB.

A database that stores data using graph structures is known as a graph database. It represents and stores data using nodes, edges, and attributes rather than tables or documents. Relationships between the nodes are represented by the edges. This makes data retrieval simpler and, in many circumstances, only requires one action. Additionally, it works fantastically as a database for fast, threaded data structures like those used on Twitter

How does a Graph Database Work?

Graphs, which are not relational databases, rely heavily on the idea of multi-relational data "pathways" for their functionality. However, the structure of graph databases is typically simple. They are largely made up of two elements:

- **The Node:** This represents the actual data itself. It may be the number of people who watched a video on YouTube; it could be the number of people who read a tweet; or it could even be fundamental information like people's names, addresses, and other such details.
- **The Edge:** This clarifies the real connection between the two nodes. It is interesting to note that edges can also have their own data, such as the type of connection between two nodes. Similar to edges, mentioned directions may also describe the direction in which the data is flowing.

Graph databases are mostly utilized for studying relationships. For instance, businesses might extract client information from social media using a graph database. For example, some organizations might use a graph database to extract data about relationships between Person, Restaurant, and City, as shown in Figure 11.2.

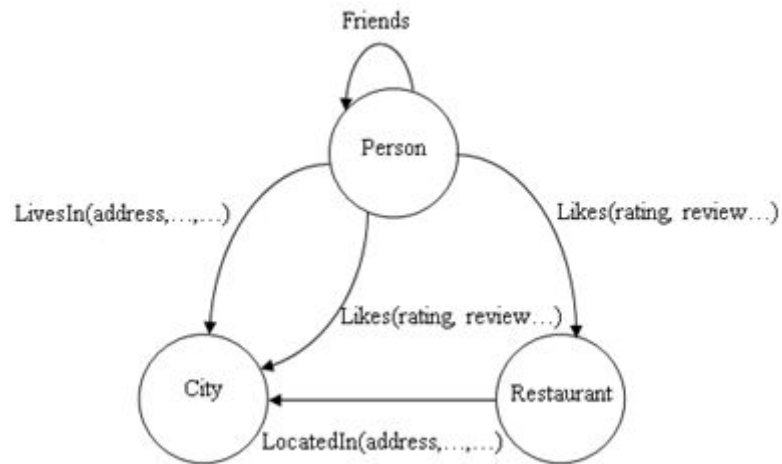


Figure 11.2: Different Nodes and Edges in Graph Database.
(Adapted from <https://www.kdnuggets.com/>)

When do we need a Graph Database?

- 1) It resolves issues with many-to-many relationships. For example, many-to-many relationships include friends of friends.
- 2) When connections among data pieces are more significant. For example, there is a profile with some unique information, but the main selling point is the relationship between these different profiles, which is how you get connected inside a network.
- 3) Low latency with big amounts of data. The relational database's data sets will grow significantly as you add more relationships, and when you query it, its complexity will increase and it will take longer than usual. However, graph databases are specifically created for this purpose, and one can easily query relationships.

Now, let's look at a more specific illustration to explain a group of people's complicated relationships. For example, five friends share a social network. These friends are Binny, Bhawna, Chaitaya, Manish, and Mohit. Their personal data may be kept in a graph database that resembles this, as shown in Figure 11.3 and Table 11.3:

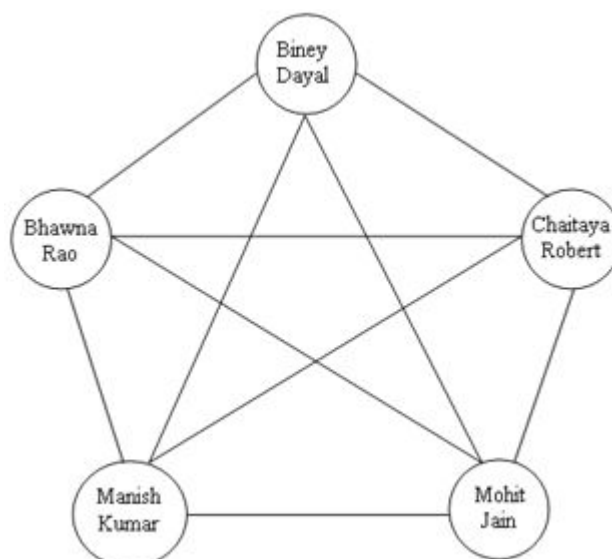


Figure 11.3: Example-Five friends sharing social network.

Table 11.3: Relational database: an example

Id	Firstname	Last Name	Email	Mobile
1001	Biney	Dayal	binnya@example.com	8645212321
1002	Bhawna	Rao	bhawanrao@example.com	9645212323
1003	Chaitaya	Robert	chaitayarob@example.com	7645212356
1004	Manish	Kumar	mkumar@example.com	9955212320
1005	Mohit	Jain	mjain@example.com	9945212329

This means we will need yet another table to keep track of user relationships. Our friendship table (refer Table 11.4) will resemble the following:

Table 11.4: Friendship Table

user_id	friend_id
1001	1002
1001	1003
1001	1004
1001	1005
1002	1001
1002	1003
1002	1004
1002	1005
1003	1001
1003	1002
1003	1004
1003	1005
1004	1001
1004	1002
1004	1003
1004	1005
1005	1001
1005	1002
1005	1003
1005	1004

We won't go too deeply into the theory of the database's main key and foreign key. Instead, presume that the friendship table uses both friends' ids. Let's say that every member on our social network gets access to a feature that lets

them view the personal information of their other users who are friends with them. This means that if Chaitaya were to ask for information, it would be regarding Biney, Bhawna, Manish and Mohit. We shall address this issue in a conventional (relational database) manner. First, we need to locate Chaitaya's user id in the database's Users table (refer Table 11.5).

Table 11.5: Chaitaya's Record

Id	First Name	Last Name	Email	Mobile
1003	Chaitaya	Robert	chaitanyarob@example.net	7645212356

We would now search the friendship table (refer Table 6) for all tuples with the user id of 3. The resulting relationship would look like this:

Table 11.6: Friendship Table for user id 3

user_id	friend_id
1003	1001
1003	1002
1003	1004
1003	1005

Let us now examine the time required for this Relational database strategy. This will be close to $\log(N)$ times, where N is the number of tuples in the friendship table. In this case, the database continues to keep the entries in sequential order based on their ids. So, in general, the time complexity for 'M' number of queries is $M \cdot \log(N)$. Only, if we had used a graph database strategy the overall time complexity would have been $O(N)$. For the simple reason that once Chaitaya has been located in the database, all the rest of her friends may be found with a single click, as shown in Figure 11.4.

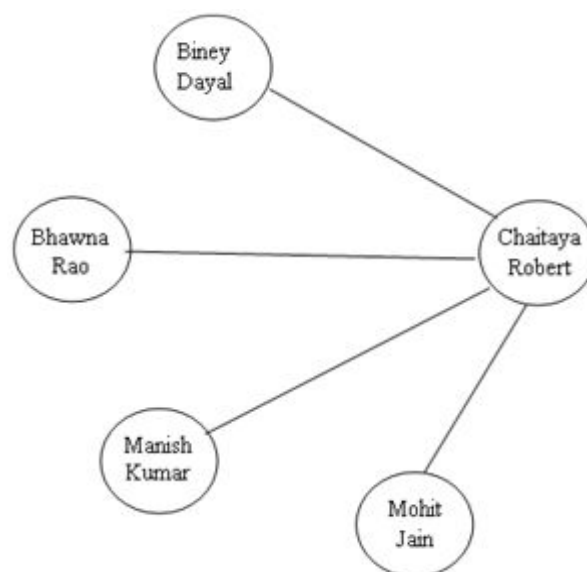


Figure 11.4: Accessing others data with a single click

Graph Database Examples

Although graph databases are not as widely used as other NoSQL databases, there are a handful that have become de facto standards when discussing NoSQL:

Neo4j is both an open-source and an interestingly developed Java graph database. It is considered to be one of the best graph databases. In addition to that, it comes with its own language known as Cypher, which is comparable to the declarative SQL language but is designed to work with graphs. In addition to Java, it supports a number of other popular programming languages, including Python, .NET, JavaScript, and a few others. Neo4j excels in applications such as the administration of data centers and the identification of fraudulent activity.

RedisGraph is a graph module that is integrated into Redis, which is a key-value NoSQL database. RedisGraph was developed to have its data saved in RAM for the same reason that Redis itself is constructed on in-memory data structures. As a result, a graph database with excellent speed and quick searching and indexing is created. RedisGraph also makes use of Cypher, which is ideal if you're a programmer or data scientist looking for greater database flexibility. Applications that require blazing-fast performance are the main uses.

OrientDB It is interesting to note that OrientDB supports graph, document store, key-value store, and object-based data formats. Having stated that, the graph model, which uses direct links between databases, is used to hold all of the relationships. Although it does not use Cypher, OrientDB is open-source and developed in Java, just like Neo4j and the two prior graph databases. OrientDB is designed to be used in situations when many data models are necessary, and as a result, it is optimized for data consistency as well as minimizing data complexity.

11.3.3 Key-value Pair Based

Key-value stores are perhaps the most widely used of the four major NoSQL database formats because of their simplicity and quick performance. Let us examine key-value stores' operation and application in more detail. With some of the most well-known platforms and services depending on them to deliver material to users with lightning speed, NoSQL has grown in significance in our daily lives. Of course, NoSQL includes a range of database types, but key-value store is unquestionably the most used.

Because of its extreme simplicity, this kind of data model is built to execute incredibly quickly when compared to relational databases. Furthermore, because key-value stores adhere to the scalable NoSQL design philosophy, they are flexible and simple to set up.

How Does a Key-Value Work?

In reality, key-value storage is quite simple. A value is saved with a key that specifies its location, and a value can be pretty much any piece of data or information. In reality, this design idea may be found in almost every programming language as an array or map object, refer Figure 11.5. The fact

that it is persistently kept in a database management system makes a difference in this case.

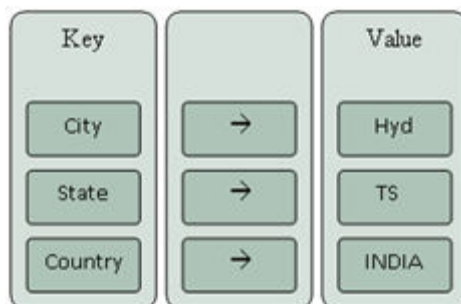


Figure 11.5: Example Key-Value database.

Popularity of key-value stores is due to the fact that information is stored as a single large piece of data instead of as discrete data. As a result, indexing the database is not really necessary to improve its performance. Instead, because of the way it is set up, it operates more quickly on its own. Similar to that, it mostly uses the get, put, and delete commands rather than having a language of its own.

Of course, this has the drawback that the data you receive in response to a request is not screened. Under certain conditions, this lack of data management may be problematic, but generally speaking, the trade-off is worthwhile. Because key-value stores are both quick and reliable, the vast majority of programmers find ways to get around any filtering or control problems that may arise.

Benefits of Key-Value

Key-value data models, one of the more well-liked types of NoSQL data models, provide many advantages when it comes to creating a database:

Scalability: Key-value stores, like NoSQL in general, are infinitely scalable in a horizontal fashion, which is one of its main advantages over relational databases. This can be a huge advantage for sophisticated and larger databases compared to relational databases, where expansion is vertical and finite, as shown in Figure 11.6.

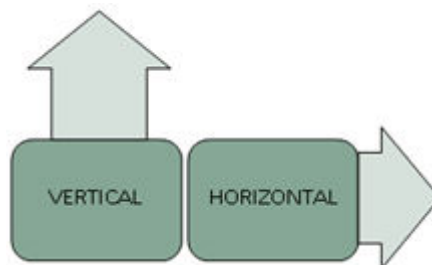


Figure 11.6: Horizontal and Vertical Scalability

More specifically, partitioning and replication are used to manage this. Additionally, by avoiding things like low-overhead server calls, it decreases the ACID guarantees.

No/Simpler Querying: With key-value stores, querying is really not possible except in very particular circumstances when it comes to querying keys, and

even then, it is not always practicable. Because there is just one request to read and one request to write, key-value makes it easier to manage situations like sessions, user profiles, shopping carts, and so on (due to the blob-like nature of how the data is stored). Similar to this, concurrency problems are simpler to manage because only one key needs to be resolved.

Mobility: Because key-value stores lack a query language, it is simple to move them from one system to another without modifying the architecture or the code. Thus, switching operating systems is less disruptive than switching relational databases.

When to Use Key-Value

Key-value stores excel in this area because traditional relational databases are not actually designed to manage a large number of read/write operations. Key-value can readily scale to thousands of users per second due to its scalability. Additionally, it can easily withstand lost storage or data because of the built-in redundancy.

As a result, key-value excels in the following instances:

- Profiles and user preferences
- Large-scale user session management
- Product suggestions (such as in eCommerce platforms)
- Delivery of personalized ads to users based on their data profiles
- Cache data for infrequently updated data

There are numerous other circumstances where key-value works nicely. For instance, because of its scalability, it frequently finds usage in Big data research. Similar to how it works for web applications, key-value is effective for organizing player sessions in MMOG (massively multiplayer online game) and other online games.

Key-Value Database Examples

Some key-value database models, for instance, save information to a solid-state drive (SSD), while others use random-access memory (RAM). We depend on key-value stores on a daily basis in our lives since they are some of the most popular and frequently used databases. The fact is that some of the most popular and commonly used databases are key-value stores.

Amazon DynamoDB is most likely the database that is used the most often for key-value storage. In point of fact, study into Amazon DynamoDB was the impetus for the rise in popularity of NoSQL.

Aerospike is a free and open-source database that was designed specifically for use with in-memory data storage.

Berkeley DB: Another free and open-source database, Berkeley DB is a high-performance framework for storing databases, despite the fact that it has a very simple interface.

Couchbase: Text searches and querying in a SQL-like format are both possible with Couchbase, which is an interesting feature.

Memcached not only saves cached data in RAM, which helps websites load more quickly, but it is also free and open source.

Riak was designed specifically for use in the app development process, and it plays well with other databases and app platforms.

Redis: A database that serves as both a memory cache and a message broker.

11.3.4 Document Based

A non-relational database that stores data as structured documents is known as a document database (also known as a NoSQL document store). Instead of using standard rows and columns, JSON format is a more recent technique to store data. An XML or JSON file, or a PDF, are all examples of documents. NoSQL is everywhere nowadays; just look at Twitter and its use of FlockDB or Amazon and their use of DynamoDB. Figure 11.7 shows the difference between the Relational and Document Store model.

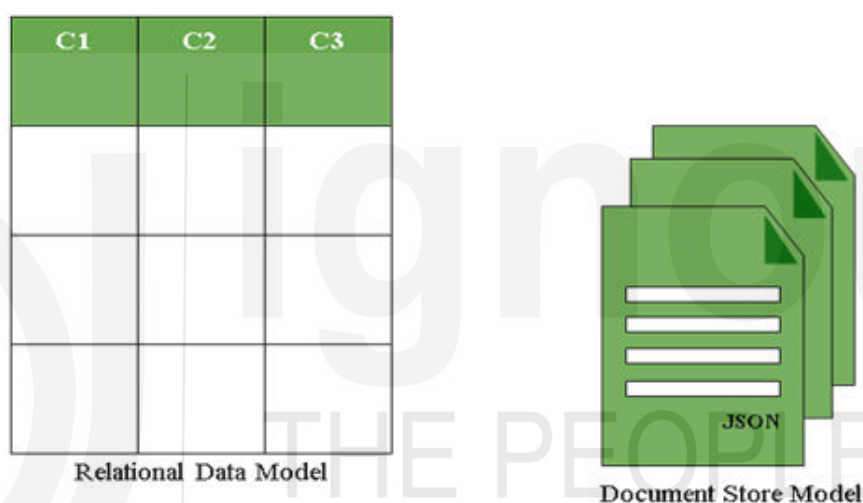


Figure 11.7: Relational Vs Document Store Model

In spite of the fact that there are a great deal of data models, each of which contains hundreds of databases, the one we are going to investigate today is called Document-store. One of the most common database models now in use, document-store functions in a manner that is somewhat similar to that of the key-value model in the sense that documents are saved together with particular keys that access the information. Figure 8 (a) shows the document that holds information about a book. This file is a JSON representation of a book's metadata, which includes the book's BookID, Title, Author, and Year and Figure 11.8 (b) shows the same metadata for Key value database.

A Document	Key	Value
{ "BookID": "978-1449396091", "Title": "DBMS", "Author": "Raghu Ramakrishnan", "Year": "2022", }	BookID	978-1449396091
	Title	DBMS
	Author	Raghu Ramakrishnan
	Year	2022

Figure 11.8: Example of Document and Key value database

When to use a document database?

- When your application requires data that is not structured in a table format.
- When your application requires a large number of modest continuous reads and writes and all you require is quick in-memory access.
- When your application requires CRUD (Create, Read, Update, Delete) functionality.
- These are often adaptable and perform well when your application has to run across a broad range of access patterns and data kinds.

How does a Document Database Work?

It appears that document databases work under the assumption that any kind of information can be stored in a document. This suggests that you shouldn't have to worry about the database being unable to interpret any combination of data types. Naturally, in practice, most document databases continue to use some sort of schema with a predetermined structure and file format.

Document stores do not have the same foibles and limitations as SQL databases, which are both tubular and relational. This implies that using the information at hand is significantly simpler and running queries may also be much simpler. Ironically, you can execute the same types of operations in a document storage that you can in a SQL database, including removing, adding, and querying.

Each document requires a key of some kind, as was previously mentioned, and this key is given to it through a unique ID. This unique ID processed the document directly instead of being obtained column by column.

Document databases often have a lower level of security than SQL databases. As a result, you really need to think about database security, and utilizing Static Application Security Testing (SAST) is one approach to do so. SAST, examines the source code directly to hunt for flaws. Another option is to use DAST, a dynamic version that can aid in preventing NoSQL injections.

Document database advantages

One major benefit of document-store is that all of the data is stored in a single location, rather than being spread out over many interconnected databases. As a result, if you do not employ relational processes, you perform better than a SQL database.

- **Schema-less:** Because there are no constraints on the format and structure of data storage, they are particularly effective at keeping huge quantities of existing data.
- **Faster creation of document and maintenance:** The creation of a document is a fairly straightforward process, and apart from that, the upkeep requirements are virtually nonexistent.
- **Open formats:** It offers a relatively easy construction process that makes use of XML, JSON, and other formats.

- **Built-in versioning:** Because it contains built-in versioning, it means that when the documents expand in size, there is a possibility that they will also expand in complexity. Versioning makes conflicts less likely.

More precisely, document stores are excellent for the following applications because schema can be changed without any downtime or because you could not know future user needs:

- eCommerce giants (Like Amazon)
- Blogging platforms (such as Blogger, Tumblr)
- CMS (Content management systems) (Like WordPress, windows registry)
- Analytical platforms (such as Tableau, Oracle server)

Document databases' drawbacks

- **Weak Atomicity:** Multi-document ACID transactions are not supported. We will need to perform two different queries, one for each collection, in order to handle a change in the document data model involving two collections. This is where the atomicity criteria are violated.
- **Consistency Check Limitations:** A database performance issue may arise from searching for documents and collections that aren't linked to an author collection.
- **Security:** In today's world, many online apps do not have enough security, which in turn leads to the disclosure of critical data. Thus, web app vulnerabilities become a cause for concern.

Document databases examples

- One of the best NoSQL database engines is **MongoDB**, which is not only well-known but also uses JSON like format. It has its own query language.
- A search engine built on the document-store data architecture is **Elasticsearch**. Database searching and indexing may be accomplished using this straightforward and easy-to-learn tool.
- **CouchDB:** In addition to Ubuntu, it also works with the social networking site Facebook. It utilizes Javascript and is developed in the Erlang programming language.
- **BaseX** is a simple, open-source, XML-based DBM that makes use of Java.

☞ Check Your Progress 2

- 1) How Does a Column Database Work? Discuss.

.....

.....

.....

.....

2) What are the different Graph Database Examples?

.....

3) Explain document based NoSQL database.

.....

11.4 INTRODUCTION to BIG DATA

In this modern era of Information and knowledge, terabytes of data are produced by a wide variety of sources every day. These sources include social media; the data of companies, including production data, customer data, financials etc.; the interactions of users; the data created by sensors and data produced by electronic devices such as mobile phones and automobiles, amongst others. This voluminous increase in data relates to the field of Big Data. This concept of "Big Data" applies to the rapidly expanding quantity of data that is being collected, and it is described primarily by having the following characteristics: volume, velocity, veracity, and variety.

The process of deriving useful information and insights from massive amounts of data is referred to as "Big data analytics". It requires the application of technologically advanced tools and procedures. "Big data architecture" refers to the pattern or design that describes how this data is received from many sources, processed for further ingestion, assessed, and eventually made available to the end-user.

The building blocks of Big data analytics are found in the Big data architecture. In most cases, the architecture components of Big data analytics consist of four logical levels or layers, as discussed below:

- **Big Data Source Layer:** A Big data environment may manage both batch processing and real-time processing of Big data sources, like data warehouses, relational database management systems, SaaS applications, and Internet of Things devices. This layer is referred to as the Big data sources layer.
- **The Management and Storage Layer:** It is responsible for receiving data from the source, converting that data into a format that the data analytics tool can understand, and storing the data in accordance with the format in which it was received.
- **Analysis Layer:** This layer is where the business intelligence that was acquired from the Big data storage layer is analysed.
- **The Consumption Layer:** It is responsible for collecting the findings from the Big Data Analysis Layer and delivering them to the appropriate Business Intelligence Layer.

The architecture components of Big data analytics typically include four logical layers or levels (discussed above), which perform fundamental processes as given below :

- **Connecting to Data Sources:** Connectors and adapters can connect to a wide range of storage systems, protocols, and networks. They can also connect to any type of data format.
- **Data Governance:** includes rules for privacy and security that work from the time data is taken in until it is processed, analysed, stored, and deleted.
- **Systems Management:** Modern Big data architectures are usually built on large-scale, highly scalable clusters that are spread out over a wide area. These architectures must be constantly monitored using central management consoles.
- **Protecting Quality of Service:** The Quality of Service framework helps define data quality, compliance policies, and how often and how much data should be taken in.

In this Unit, we will discuss Big data, its Characteristics and Applications. We will also discuss Big data architecture and related technologies such as Map Reduce, HDFS, Apache Hadoop and Apache YARN.

11.4.1 Big Data and Characteristics

To define the concept of Big data, we first revise the fundamentals of data, and thereafter we will extend our discussion to the advanced concepts of Big data. What exactly does "Data" mean? Data can be numbers or characters or symbols or any other form such as an image or signal, which can be processed by a computer or may be stored in the storage media or sent in the form of electrical impulses.

Now, we define - What is Big data? The term "Big data" refers to a collection of information that is not only extremely large in quantity but also growing at an exponential rate over the course of time. Because of the vast amount and the high level of complexity, none of the standard methods that are used for managing data can effectively store or deal with such data. Simply speaking, "Big data" refers to information that is kept in exceedingly vast amounts.

It is vital to have a thorough understanding of the characteristics of Big data in order to get a handle on how the concept of Big data works and how it can be used. The **characteristics of Big data** are:

1. **Volume:** The amount of information that you currently have access to is referred to as its volume. We measure the quantity of data that we have in our possession using the units of Gigabytes (GB), Terabytes (TB), Petabytes (PB), Exabytes (EB), Zettabytes (ZB), and Yottabytes (YB). As per the present trends, the volume of Big data is increasing exponentially.
2. **Velocity:** The speed at which data is generated and, therefore, should be processed is referred to as the "velocity" of that process. A high velocity is a fundamental requirement for the efficiency of any operation involving vast amounts of data.

3. **Variety:** The term "variety" relates to the numerous forms that Big data might take. When we talk about diversity, we mean the various kinds of Big data that are available. This is a major problem for the Big data sector since it has an impact on the efficiency of analysis. It is critical that you organise your data in order to manage its diversity effectively. You get a wide range of data from a variety of sources.
4. **Veracity:** The veracity of data relates to its accuracy. Lack of veracity can cause significant harm to the precision of your findings; it is one of the Big data characteristics that is considered to be of the utmost importance.
5. **Value:** The benefits that your organisation gains from utilising the data are referred to as "value." Are the results of Big data analysis in line with your company's goals? Is it helping your organisation expand in any way? In Big data, this is a vital component.
6. **Validity:** This characteristic of Big data relates to how valid and pertinent the facts are for the purpose they were designed for.
7. **Volatility:** Volatility relates to the life span of Big data. Some data items may be valid for a very short duration, like the sentiments of people.
8. **Variability:** The field of Big data is continually changing. Sometimes, the information you obtained from one source may not be the same as what you found today. This phenomenon is known as data variability, and it has an impact on the homogeneity of your data.
9. **Visualisation:** You may be able to express the insights that were generated by Big data by using visual representations such as charts and graphs. The insights that Big data specialists have to provide are increasingly being presented to audiences who are not technically oriented.

Some examples of Big data are given below:

- Big data can be illustrated by looking at the Stock Exchange, which creates approximately one terabyte of new trade data every single day.
- Social Media: According to various reports, more than 500 Gigabytes of fresh data are added to the databases of the social media website each and every day. The primary sources of this data are the uploading of photos and videos, sending and receiving of messages, posting of comments, and other activities.
- In just thirty minutes of flying time, a single jet engine is capable of producing more than ten Gigabytes of data. Because there are many thousands of flights each day, the amount of data generated can reach many Peta bytes.

A Big data system is primarily an application of Big data in an organisation for decision support. The following four components are required for a Big data system to function properly:

1. Ingestion (collecting and preparing the data)

2. Storage (storing the data)
3. Analysis (analysing the data)
4. Consumption (presenting and sharing the insights)

You need a component that is responsible for collecting the data and another that is responsible for storing it. In order for your Big data ecosystem to be complete, you require a system that analyses and reports the results of Big data analysis.

Big data may be the future of worldwide governance and businesses. It presents businesses with a great number of opportunities for improvement. The following are some of the more important ones:

1. Improved Decision Making
2. Data-driven Customer Service
3. Efficiency Optimisation
4. Real-time Decision Making

Based on these opportunities, this field of Big data has enormous applications in a variety of fields, and they are discussed in the subsequent section, i.e. section 5.4.

Check Your Progress 3

1. What does "Big Data" mean?
.....
.....
.....
2. List the components required by a Big data system to function properly
.....
.....
.....
3. Discuss the essential characteristics of Big data.
.....
.....
.....

11.4.2 Big Data Applications

It is generally stated that Big data is one of the most valuable and effective fuel that can power the vast IT companies of the 21st century. Big data has applications in virtually every industry. Big data enables businesses to make better use of the vast quantities of data they generate and collect from a variety of sources. There are many different applications for Big data, which is why it is currently one of the skills that is most in demand. The following are some examples of important applications of Big data:

- One of the industries that makes the most use of Big data technology is the travel and tourism sector. It has made it possible for us to anticipate the need for travel facilities in a variety of locations, thereby improving business through dynamic pricing and several other factors.
- The social media sites produce a significant amount of data. Big data allows marketers to make greater use of the information provided by social media platforms, which results in improved promotional activities. It enables them to construct accurate client profiles, locate their target audience, and comprehend the criteria that are important to them.
- Big data technology is utilised widely within the financial and banking sectors. The use of Big data analytics can assist financial institutions in better comprehending the behaviours of their clients on the basis of the inputs obtained from their investment behaviours, shopping habits, reasons for investing, and personal or financial histories.
- The field of healthcare has already witnessed significant changes brought on by Big data application implementation. Individual patients can now receive individualised medical care tailored to their specific needs. This is possible due to the application of predictive analytics by medical professionals and personnel in the healthcare industry.
- The use of Big data in recommendation systems is another popular application of Big data. Big data allows businesses to recognise patterns of client behaviour in order to provide better and more individualised services to those customers.
- One of the most important industries that produce and use Big data is the telecommunications and multimedia business. Every day, Zettabytes of new data are created and managed.
- In addition, the Government and the military make heavy use of Big data technology. You may think of the quantity of data that the Government generates on its records, and in the military, a standard fighter jet plane needs to handle Petabytes of data while it is in the air.
- Companies are able to do predictive analysis because of the capabilities provided by Big data technologies. It lets businesses make more accurate predictions of the outcomes of processes and events, which in turn helps them reduce risk.
- Companies are able to develop insights that are more accurate because of Big data. They collect big data from a variety of sources, which allows them the capacity to use relevant data to generate insights that can be put into action. If a corporation has more accurate information, it will be able to make decisions that are more profitable and reduce risks.

11.4.3 Structured, Semi-Structured and Unstructured Data

The terms "structured," "unstructured," and "semi-structured" data are frequently brought up whenever we are having a discussion about data or analytics. These are the three varieties of data that are becoming increasingly important for many kinds of commercial applications, i.e. Structured, Semi-

Structured and Unstructured data. Structured data has been around for quite some time, and even today, conventional systems and reporting continue to rely on this type of data. In spite of this, there has been a rapid increase in the production of unstructured and semi-structured data sources over the course of the last several years. As a consequence of this, an increasing number of companies are aiming to include all three kinds of data in their business intelligence and analytics systems in order to take these systems to the next level.

In this section of the Unit, we will discuss data types used for understanding and implementing Big data, i.e. Structured, Semi Structured and Unstructured data.

Structured Data: Structured data is a term that refers to information that has been reformatted and reorganised in accordance with a data model that has been decided in advance. After mapping the raw data into the predesigned fields, the data can be extracted and read using SQL in a straightforward manner. Relational databases, which are characterised by their organisation of data into tables consisting of rows and columns, and the query language supported by them (SQL) provide the most apparent example of structured data.

This relational model of the data format decreases the quantity of duplicated information. On the other side, organised data is more dependent on one another and is less flexible than unstructured data. Both humans and machines are responsible for the generation of this kind of data.

There are several examples of structured data that are generated by machines, such as data from point-of-sale terminals (POS), such as quantity, barcodes, and statistics from blogs. In a similar vein, anybody who works with data has probably used spreadsheets at least once in their life. Spreadsheets are an example of a classic form of structured data that is generated by humans. Because of the way it is organised, structured data is simpler to examine compared to semi-structured data as well as unstructured data. Structured data can be analysed easily because it corresponds to data models that have already been defined. For example, you can organise structured data such as names of customers in alphabetical order, manage telephone numbers in the appropriate format, organise social security numbers in the correct format etc.

Unstructured data: Information displayed in its most unprocessed form is referred to as "unstructured data". It is really difficult to work with this data because it has a minimal structure, and the formatting is also very confusing. Unstructured data management can collect data from a variety of sources, such as postings on social media platforms, conversations, satellite imagery, data from the Internet of Things (IoT) sensor devices, emails, and presentations, and organise it in a storage system in a logical and specified manner.

Semi-Structured Data: There is a third kind of data that falls between structured and unstructured data, called semi-structured data or partially structured data. Your data sets might not always be structured or unstructured. One variety of such data is known as semi-structured data, and

it differs from unstructured data in that it possesses both consistent and definite qualities. It does not restrict itself to a fixed structure like those required for relational databases. Although data organisational features such as metadata or semantic tags are utilised with semi-structured data to make it more manageable, there is still a certain amount of unpredictability and inconsistency in the data.

The use of delimited files is one illustration of a data format that is only semi-structured. It has elements capable of separating the data hierarchies into their own distinct structures. In a similar manner, digital images have certain structural qualities that make them semi-structured, but the image itself does not have a pre-defined structure. If a picture is shot with a Smartphone, it will include certain structured properties like a device ID, a geotag, and a date and time stamp. After they have been saved, pictures can be organised further by affixing tags to them, such as "pet" or "dog," for example.

Due to the presence of one or more characteristics that allow for classification, unstructured data may occasionally be categorised as semi-structured data instead. To summarize, organizations need to analyze all three kinds of data in order to stay ahead of the competition and make the most of the knowledge they generate.

☞ Check Your Progress 4

1. Give Five applications of Big data.

.....

.....

.....

2. Compare Structured, Semi-Structured and Unstructured Data.

.....

.....

.....

3. Discuss the role of Structured, Semi-Structured and Unstructured Data in decision-making.

.....

.....

.....

11.4.4 Big Data Vs Data Warehouse

What should you buy for implementing an analytics system for your organization - a Big data solution or a data warehouse? A data warehouse and a Big data solution are quite similar in many respects. Both have the capacity to store a significant amount of information. When reporting, either option is acceptable to utilise. But the question that needs to be answered is whether they can truly be utilised as a substitute for each other. In order to understand this, you need to have a conceptual understanding of both, i.e., Big data and Data Warehouse.

The form of Big data that can be found in Hadoop, Cloudera, and other similar Big data platforms is the type that most people understand. The following are the working definitions of Big data solutions (You may find an accurate functioning definition on the websites of Cloudera or Horton Works.):

- A type of technology that is able to store extremely vast volumes of data.
- A technology capable of storing data in devices that are not pricey.
- A type of data storage technology in which the data is kept in an unstructured manner.

The data warehouse has had widespread usage in the past two decades, whereas Big data applications have seen a significant rise over the course of the past decade. Big data is expected to eventually replace traditional data warehousing, which is the first thing that comes to the mind of someone who is not technically very deep into these technologies. The fact that they share similarities is another argument favouring this simplistic way of thinking. Following are some of the similarities between the two:

- Both of these have a significant amount of information.
- Both may be utilised in the reporting process.
- Both are controlled by devices that save information electronically.

But despite this, "Big Data" and "Data Warehouse" are not the same thing at all. Why? In order to understand this, we need to recapitulate the basic concepts of Data Warehouse

A data warehouse is a subject-oriented, non-volatile, integrated, and time-variant collection of data generated to help the management make business decisions. A data warehouse provides a centralised, integrated, granular, and historical point of reference for all of the company's data in one convenient location.

So why do individuals demand a solution that involves Big data? People are looking for a solution for Big data because many organizations have a significant amount of data. In such companies, that data – if it is accessed and processed properly – can include a great deal of important information that can lead to better decisions, which in turn can lead to more revenue, more profitability, and more customers. Thus, a vast majority of businesses want to analyse Big data.

What are the reasons that people require a data warehouse? In order for people to make decisions based on accurate information, a data warehouse is required. You need data that is trustworthy, verifiable, and easily accessible to everyone if you want to have a thorough understanding of what is occurring within your company.

So, in short, we can understand the data Warehouse involves collecting data from a number of sources, processing it, and storing it in a repository where it can be analyzed and used for reporting reasons—the procedure described above results in the establishment of a data warehouse.

Comparison: Big Data Vs Data Warehouse

A Big data solution is an example of technology, whereas data warehousing is an example of an architectural framework. Technology can be loosely defined as any mechanism that is capable of storing and organising huge amounts of data. A facility that stores and organises data with the aim of guaranteeing a company's credibility and integrity is known as a data warehouse. Data warehouses are becoming increasingly common. When data is taken from a data warehouse, the person taking the data is aware that other individuals are using the same data for various purposes.

Table 11.7: Big Data Vs Data Warehouse

S.No.	Big Data	Data Warehouse
1.	Big data refers to a very large amount of data that can be utilised by many technologies.	The term "data warehouse" refers to the accumulation of historical data from a variety of business processes within an organisation.
2.	The term "Big Data" refers to a type of technology that can store and manage extensive amounts of data.	A data warehouse is a structure that is utilised in the process of data organisation.
3.	As input, it will accept data that is either structured, non-structured, or semi-structured.	The only type of data that can be used as input is structured data.
4.	The distributed file system is used for processing Big data.	Processing operations in the data warehouse are not carried out using a distributed file system.
5.	When it comes to retrieving data from Big data stores, Big data may not use SQL queries.	To get data from relational databases, we use structured query language, or SQL, in the data warehouse.
6.	Apache Hadoop is capable of managing massive amounts of data when handled appropriately.	It is difficult for a data warehouse to manage an extremely large volume of unstructured data.
7.	The modifications to the data that occur as a result of adding new data are saved in the form of a file.	Changes in the data that occur as a result of adding new data do not have an immediate and direct effect on the data warehouse.
8.	When opposed to data warehouses, Big data does not require the same level of effective management strategies.	As a result of the fact that the data is compiled from a variety of business divisions, the data warehouse calls for more effective management strategies.

Big data uses a Distributed File System. So, we discuss the Distributed File Systems in section 11.4.5 of this Unit.

☞ Check Your Progress 5

1. Give similarities between Big Data and Data of a Data Warehouse.

.....

.....

.....

2. Compare Big Data and Data Warehouse.

.....

.....

.....

11.4.5 Distributed File System

A Distributed File System (DFS) is a file system that is distributed across many file servers at several locations (It's possible that these servers are located in various parts of the world). It allows programmers to access or store isolated files the same way as local files, allowing them to access files from any network or computer.

The primary objective of the Distributed File System (DFS) is to facilitate the sharing of users' data and resources between physically separate systems through the utilisation of a Common File System. A setup on DFS is defined as a group of workstations and mainframes that are linked together over a Local Area Network (LAN). Within the framework of the operating system, a DFS is carried out. A namespace is generated by DFS, and the clients are not exposed to the inner workings of this generation process.

The DFS is comprised of two distinct components, given below:

- **Location transparency:** A namespace component is a useful tool for achieving Location transparency.
- **Redundancy:** It is established by the utilisation of a file replication component.

Together, these components make it more likely that data will be accessible even if there is a breakdown or a severe load, as redundant data is stored in multiple locations that are shared under a single folder referred to as the "DFS root."

It is possible to use the namespace component of Distributed File System (DFS) without using the file replication component, and it is perfectly possible to use the file replication component of Distributed File System (DFS) between servers without using the namespace component. It is not essential to make use of both of the DFS components at the same time.

In order to gain more clarity on the Distributed File System (DFS), we need to study the Features of DFS:

1. Transparency:

When discussing distributed systems, the term "transparency" refers to the process of hiding information about the separation of components from both the user and the application programmer. Transparency protects the integrity of the system. It makes the entire system a single system rather than a collection of distinct components working together. It is classified into the following types:

- a) **Structure Transparency** – There is no reason for the client to be aware of the number of file servers and storage devices and their geographical locations. But it is always recommended to provide a number of different file servers to improve performance, adaptability, and dependability.
 - b) **Access Transparency** – The access process for locally stored files and remotely stored files should be identical. It should be possible for the file system to automatically locate the file that is being accessed and send that information to the client's side.
 - c) **Naming transparency** – There should be no indication in the name of the file as to where the file is located in any way, shape, or form. Once a name has been assigned to the file, it should not have any changes made to it while it is being moved from one node to another.
 - d) **Replication transparency** – If a file is copied on multiple nodes, the locations of the copies of the file as well as the copies themselves, should be hidden when moving from one node to another.
2. **User Mobility:** It will automatically transfer the user's home directory to the node where the user logs in.
 3. **Performance:** The average amount of time it takes to get what the client wants is used to measure performance. This time includes CPU time, the time it takes to get to secondary storage, and the time it takes to get to the network. It would be best for the Distributed File System to work like both DFS and as well as a centralised file system (CFS), based on requirements.
 4. **Simplicity and ease of use:** A file system should have a simple user interface and a small number of commands in the file.
 5. **High availability:** If a link fails, a node fails, or a storage drive crashes, a Distributed File System should still be able to keep working. A distributed file system (DFS) that is both reliable and flexible should have different file servers that control different storage devices that are also separate.
 6. **Scalability:** Adding new machines to the network or joining two networks is a common way to make the network bigger, so the distributed system will always grow over time. So, a good distributed file system (DFS) should be built so that it can grow quickly as the number

of nodes and users grows. As the number of nodes and users grows, the service should not deteriorate too much.

7. **High reliability:** A good distributed file system (DFS) should make data unlikely to be lost. That is, users should not feel they have to make backup copies of their files because the system is unreliable. Instead, a file system should make copies of important files in case the originals get lost. Stable storage is used by many file systems to make them very reliable.
8. **Data integrity:** A file system is often used by more than one person at a time. The file system must make sure that the data saved in a shared file stays the same. That is, a concurrency control method must be used to keep track of all the different users' requests to access the same file at the same time. Atomic transactions are high-level concurrency management that a file system often offers the users to keep their data safe.
9. **Security:** A distributed file system should be safe so that its users can trust that their data will be kept private. Security measures must be put in place to protect the information in the file system from unwanted and unauthorized access.
10. **Heterogeneity:** Because distributed systems are so big, there is no way to avoid heterogeneity. Users of heterogeneous distributed systems can choose to use different kinds of computers for different tasks.

The working of DFS can be put into practice in one of two different ways:

- **A standalone DFS namespace** is a namespace that only permits DFS roots to be used in it if they are located on the local computer and do not make use of Active Directory. Only the computers on which a Standalone DFS was initially created can be used to acquire the file system. It offers no fault tolerance, and it cannot be connected to any other DFS. Standalone DFS roots are not very common because of their limited advantage.
- **Domain-based DFS namespace** — This creates the DFS namespace root, which can be accessed at `domainname>dfsroot>` and stores the configuration of DFS in Active Directory.

Advantages of Distributed File System (DFS):

- a) DFS permits many active users to access or store data simultaneously.
- b) DFS allows the remote sharing of data
- c) DFS facilitates file searching; it is faster to get to the data and makes the network run better.
- d) DFS improves the ability to change the size of the data and the ability to exchange data.
- e) DFS maintains data transparency and replicability; even if a server or disc fails, Distributed File System makes data available.

Disadvantages of Distributed File System (DFS):

- a) Because all the nodes and connections in a Distributed File System need to be secured, we can safely claim that security is at risk.
- b) During the process of moving from one node to another in the network, there is a chance that some of the messages and data will be lost.
- c) While using a distributed file system, Difficulty is encountered when attempting to connect to a database
- d) When compared to a system with a single user, a Distributed File System makes database management more difficult.
- e) If every node in the network attempts to transfer data at the same time, there is a possibility that the network will become overloaded.

After understanding the Distributed File System (DFS), now it is time to understand Hadoop and HDFS (Hadoop Distributed File System). The discussion for HDFS (Hadoop Distributed File System) and MapReduce is given in the subsequent section, and the details of MapReduce are explicitly available in Section 11.4.6.

✎ Check Your Progress 6

1. Describe the term "Distributed File System" in the context of Big data.
.....
.....
.....
2. What is the primary objective of the Distributed File System?
.....
.....
.....
3. Explain the components of the Distributed File System.
.....
.....
.....
4. Discuss the features of Distributed File System
.....
.....
.....
5. Discuss the number of ways through which the working of Distributed File System can be put into practice
.....
.....
.....

6. Give advantages and disadvantages of Distributed File Systems.

.....

.....

.....

11.4.6 HDFS and Mapreduce

Data is being produced at lightning speed in the modern period by a wide variety of sources, such as corporations, scientific data, emails, blogs, and other online platforms, amongst other places. It is necessary to implement data-intensive applications and storage clusters in order to analyze and manage the massive amount of data that is being generated and to gain information that is of use to the users. This is a prerequisite for being able to analyze and manage the data. This category of application is required to fulfill a number of characteristics, such as being able to tolerate errors, performing parallel processing, distributing data, maintaining load balance, being scalable, and having highly available operation. The MapReduce programming approach was developed by Google specifically for the purpose of resolving problems of this kind. Hadoop is another name for the open-source software project known as Apache Hadoop, which implements MapReduce technology.

Hadoop is a collection of several software services that are all freely accessible to the public and can be used in conjunction with one another. It offers a software framework for storing a huge amount of data in a variety of locations and for working with that data by utilising the MapReduce programming style. Both of these capabilities are accomplished through the use of software. Hadoop's essential components are the Hadoop Distributed File System (HDFS) and the MapReduce programming paradigm. Hadoop is operated using a programming style called MapReduce, and the Hadoop Distributed File System (HDFS) is the file system that is used to store data. The combination of HDFS and MapReduce creates an architecture that, in addition to being scalable and fault-tolerant, conceals all of the complexity associated with the analysis of Big data.

Hadoop Distributed File System abbreviated as HDFS, is a self-healing, distributed file system that offers dependable, scalable, and fault-tolerant data storage on commodity hardware. HDFS was developed by Hadoop. It collaborates closely with MapReduce by distributing storage and processing across huge clusters. This is accomplished by combining storage resources that, depending on the requests and queries being processed, are able to scale up or down. HDFS is not architecture-specific; it may read data in any format, including text, photos, videos, and so on, and it will automatically optimise it for high bandwidth streaming. The ability to tolerate errors is the most significant benefit of HDFS. The possibility of a catastrophic failure can be reduced by ensuring a quick data movement between the nodes and that Hadoop can continue to offer service even if individual nodes fail.

The information presented in this section can be used for understanding the development of large-scale distributed applications that can make use of the

computing capacity of several nodes in order to finish tasks that are data and computation intensive.

Let us discuss the Hadoop Architecture, which includes the following components:

- the MapReduce programming framework and
- the Hadoop distributed file system (HDFS)

An understanding of the Hadoop architecture requires the introduction of various daemons involved in its working. So, we need to understand the various daemons involved in Apache Hadoop, which includes the five daemons, three of which, i.e. NameNode, the DataNode, and the Secondary NameNode, relate to **HDFS** for the purpose of efficiently managing distributed storage, and The JobTracker and TaskTracker components that are utilised by the **MapReduce** engine are responsible for both job tracking and job execution respectively, and each of the mentioned daemons runs on their respective JVM

Firstly, we will discuss **HDFS**, a distributed file system comprising three nodes, i.e., the NameNode, the DataNode, and the Secondary NameNode, respectively.

- 1) **NameNode:** A single NameNode daemon operates on the master node. NameNode is responsible for storing and managing the metadata that is connected with the file system. This metadata is stored in a file that is known as fsimage. When a client requests to read from or write to a file, the metadata is held in a cache located within the main memory so that the client may access it more rapidly. The I/O tasks are completed by the slave DataNode daemons, which are directed in their actions by the NameNode.

The NameNode is responsible for managing and directing how files are divided up into blocks, selecting which slave node should store these blocks, and monitoring the overall health and fitness of the distributed file system. In addition, it decides which slave node should store these blocks. Memory and input/output (I/O) are both put to intensive use in the operations that are carried out by the NameNode in the network.

- 2) **DataNode:** A DataNode daemon is present on each slave node, which is a component of the Hadoop cluster. DataNodes are the primary storage parts of HDFS. They are responsible for storing data blocks and catering to requests to read or write files that are stored on HDFS. These are under the authority of NameNode. Blocks that are kept in DataNodes are replicated in accordance with the configuration in order to guarantee both high availability and reliability. These duplicated blocks are dispersed around the cluster so that computation may take place more quickly.
- 3) **Secondary NameNode:** A backup for the NameNode is not provided by the Secondary NameNode. The job of the Secondary NameNode is to read the file system at regular intervals, log any changes that have

occurred, and then apply those changes to the fsimage file. This assists in updating NameNode so that it can start up more quickly the next time, as shown in Figure 3.

The HDFS layer is responsible for daemons that store data and information, such as NameNode and DataNode. The MapReduce layer is in charge of JobTracker and TaskTracker, which are seen in Figure 9 and are responsible for keeping track of and actually executing jobs.

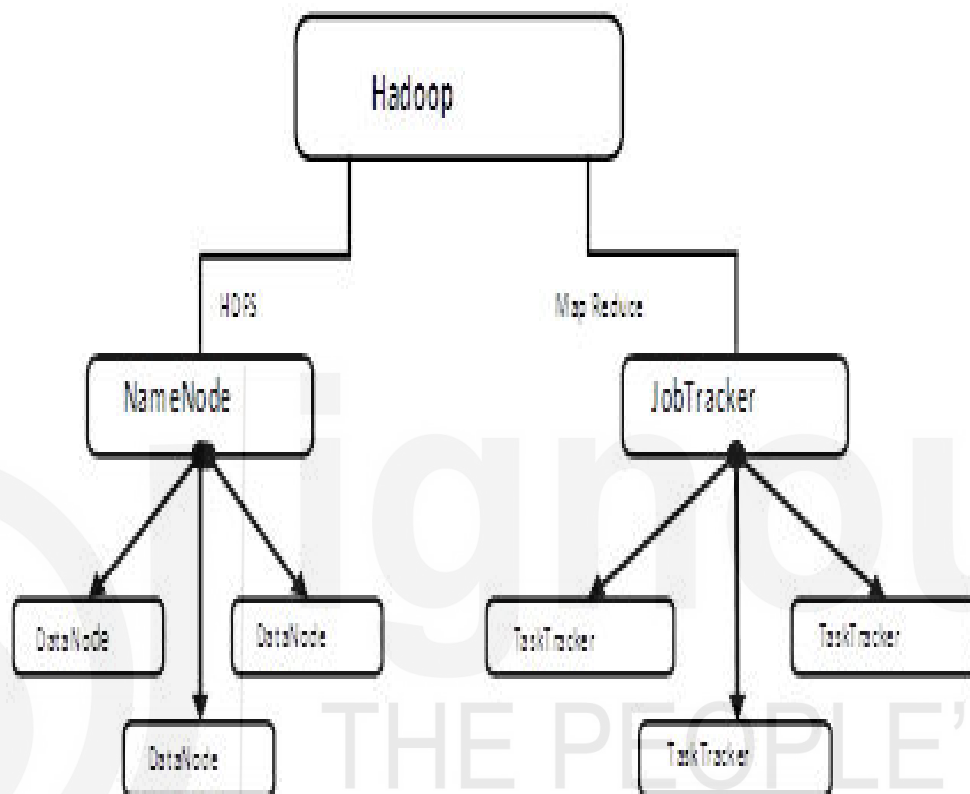


Figure 11.9: Hadoop Daemons

The master/slave architecture is utilised by HDFS, with the NameNode daemon and secondary NameNode both operating on the master node and the DataNode daemon running on each and every slave node, as depicted in Figure 11.10. The HDFS storage layer consists of three different daemons.

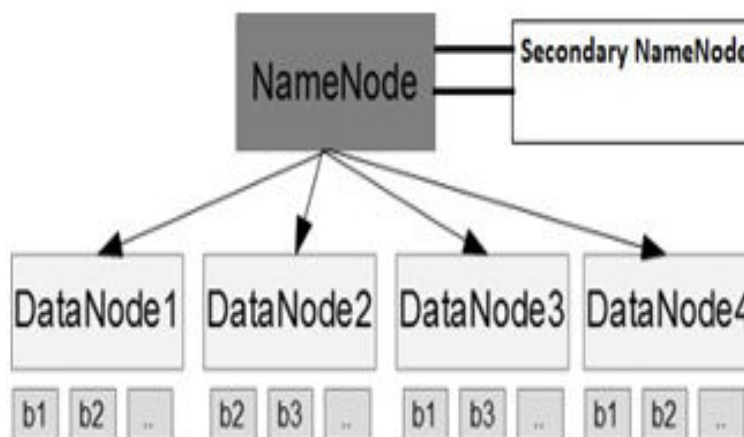


Figure 11.10: HDFS Architecture

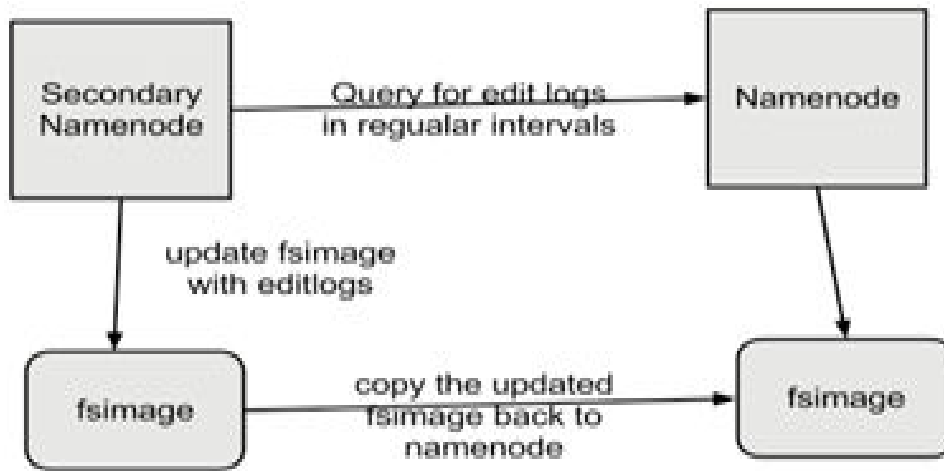


Figure 11.11: Task of Secondary name Node

A Hadoop cluster consists of several slave nodes in addition to a single master node. NameNode is the master daemon for the HDFS storage layer, while JobTracker is the master daemon for the MapReduce processing layer. Both of these daemons are executed by the master node. The remaining machines will be responsible for running the "slave" daemons, which include DataNode for the HDFS layer and TaskTracker for the MapReduce layer.

It is possible for a master node to take on the role of a slave at times, as it possesses the potential to flip roles. As a result, the master node is capable of operating both the master daemons and the slave daemons in addition to running the master daemons. The daemons that are operating on the master node are basically responsible for coordinating and administering the daemons that are running as slaves on all of the other nodes. This responsibility lies with the master node. These slave daemons are responsible for carrying out the tasks essential for the processing and storing of data.

Now we will discuss the MapReduce concept and the daemons involved with MapReduce, i.e. the JobTracker and TaskTracker components that are utilised by the **MapReduce** engine.

MapReduce can refer to either a programming methodology or a software framework. Both are utilised in Apache Hadoop. Hadoop MapReduce is a programming framework that is made available for creating applications that can process and analyse massive data sets in parallel on large multi-node clusters of commodity hardware in a manner that is scalable, reliable, and fault tolerant. The processing and analysis of data consist of two distinct stages known as the Map phase and the Reduce phase.

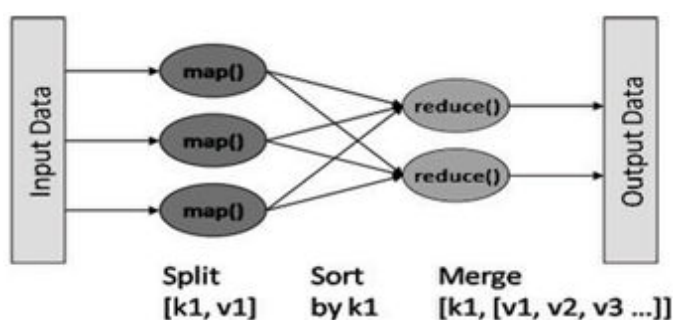


Figure 11.12: Map and Reduce Phase

During a MapReduce task, the input data is often segmented and partitioned into pieces before being broken up and processed in parallel by the "Map phase" and then by the "Reduce phase." The data that is generated by the Map phase is arranged and organised by the Hadoop architecture.

The result of the Map phase is sorted by the Hadoop framework, and this information is then sent as input to the Reduce phase in order to begin parallel reduce jobs (see Figure 12). These input and output files are saved in the system's file directory. The HDFS file system is the source of input datasets for the MapReduce framework by default. It is not required that tasks involving Map and Reduce proceed in a sequential way. This means that the Reduce jobs can begin as soon as any of the Map tasks finishes the work that has been given to them. It is also not required that all Map jobs be finished before any reduction tasks begin their work. MapReduce operates on key-value pairs as its data structure. In theory, a MapReduce job will accept a data set as an input in the form of a key-value pair, and it will only produce output in the form of a key-value pair after processing the data set through MapReduce stages. As can be seen in Figure 13, the output of the Map phase, which is referred to as the intermediate results, is sent on to the Reduce phase as an input.



Figure 11.13: MapReduce key-value pairs

On the same lines as HDFS, MapReduce also makes use of a master/slave architecture. As illustrated in Figure 14, the JobTracker daemon resides on the master node, while the TaskTracker daemon resides on each of the slave nodes.

The MapReduce processing layer consists of two different daemons, i.e. JobTracker and TaskTracker; their roles are discussed below:

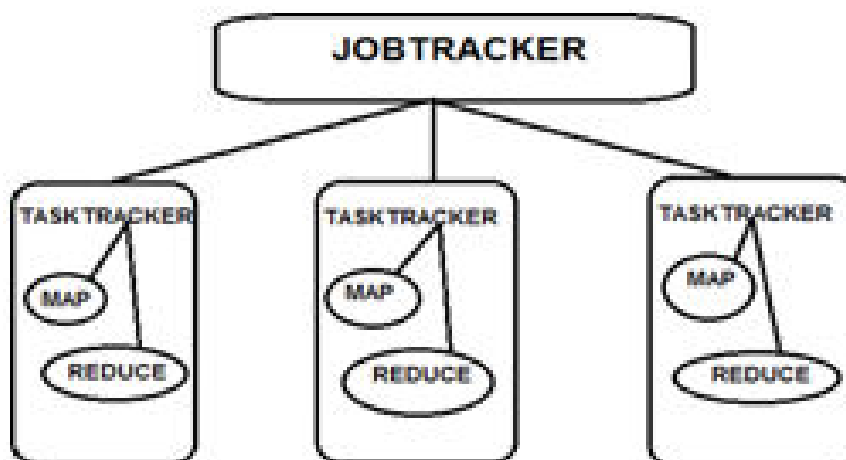


Figure 11.14: JobTracker and TaskTracker

- 4) **JobTracker:** The JobTracker service is responsible for monitoring MapReduce tasks that are carried out on slave nodes and are hosted on the master node. The job is sent to the JobTracker by the user through their interaction with the Master node. The next thing that happens is that JobTracker queries NameNode to find out the precise location of the data in HDFS that needs to be processed. JobTracker searches for TaskTracker on slave nodes and then sends the jobs to be processed to TaskTracker on those nodes. The TaskTracker will occasionally send a heartbeat message back to the JobTracker to verify that the TaskTracker on a specific slave node is still functioning and working on the task that has been assigned to it. If the heartbeat message is not received within the allotted amount of time, the TaskTracker running on that particular slave node is deemed to be inoperable, and the work that was assigned to it is moved to another TaskTracker to be scheduled. The combination of JobTracker and TaskTracker is referred to as the MapReduce engine. If there is a problem with the JobTracker component of the Hadoop MapReduce service, all active jobs will be halted until the problem is resolved.
- 5) **TaskTracker:** On each slave node that makes up a cluster, a TaskTracker daemon is executed. It works to complete MapReduce tasks after accepting jobs from the JobTracker. The capabilities of a node determine the total amount of "task slots" that are available in each TaskTracker. Through the use of the heartbeat protocol, JobTracker is able to determine the number of "task slots" that are accessible in TaskTracker on a slave node. It is the responsibility of JobTracker to assign suitable work to the relevant TaskTrackers, and the number of open task slots will determine how many jobs can be assigned. On every slave node, TaskTracker is the master controller of how each MapReduce action is carried out. Even though there is only one TaskTracker for each slave node, each TaskTracker has the ability to start multiple JVMs so that MapReduce tasks can be completed simultaneously. JobTracker, the master node, receives a "heartbeat" message regularly from each slave node's TaskTracker. This message confirms to JobTracker that TaskTracker is still functioning.

In short, it can be said that, when it comes to processing massive and unstructured data volumes, the Hadoop MapReduce computing paradigm and HDFS are becoming increasingly popular choices. While masking the complexity of deploying, configuring, and executing the software components in the public or private cloud, Hadoop makes it possible to interface with the MapReduce programming model. Users are able to establish clusters of commodity servers with the help of Hadoop. MapReduce has been modelled as an independent platform-as-a-service layer that cloud providers can utilise to meet a variety of different requirements. Users are also given the ability to comprehend the data processing and analysis processes.

11.4.7 Apache Hadoop 1 And 2 (YARN)

Apache Hadoop 1.x suffered from a number of architectural flaws, the most notable of which was a decline in the overall performance of the system. Actually, the cause of the problem was the excessive strain that was placed on the MapReduce component of Hadoop-1. MapReduce was responsible for both application management and resource management in Hadoop 1; however, with Hadoop 2, application management is now handled by a new component known as YARN, which takes over responsibility for resource management (yet another resource negotiator). As a result, MapReduce is in charge of managing application management in Hadoop 2, whereas YARN is in charge of managing the resources. With the release of Hadoop 2, YARN has added two additional daemons. These are-

- Resource Manager
- Node Manager

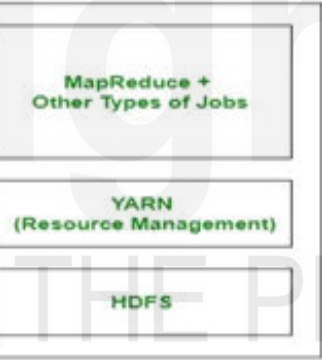
These two new Hadoop 2 daemons have replaced the JobTracker and TaskTracker in Hadoop 1.

In addition, there is only one NameNode in the Hadoop 1.x cluster, which implies that it serves as the single point of failure for the entire system. Hadoop 2.x's architecture, on the other hand, incorporates both Active and Passive NameNodes in several places. In the event that the active NameNode is unable to complete the tasks assigned to it, the passive NameNode will step in and assume responsibility. This element is directly responsible for Hadoop 2.x's high availability, which is one of the most important characteristics of the new version.

The processing of data is problematic in Hadoop 1.x; however, Hadoop 2.x's YARN provides a centralised resource management that allows sharing of sharable resources. This makes it possible for multiple applications included within Hadoop to execute concurrently while sharing a single resource.

The comparison between Apache Hadoop-1 and Hadoop-2 is consolidated below:

Comparison Parameter	Apache Hadoop 1	Apache Hadoop 2
Components	Has MapReduce	Has YARN(Yet Another Resource Negotiator) and MapReduce version 2.
Daemons	1) NameNode, 2) DataNode, 3) Secondary NameNode, 4) JobTracker, 5) TaskTracker	1) NameNode, 2) DataNode, 3) Secondary NameNode, 4) Resource Manager, 5) Node Manager

Working	Both HDFS and MapReduce are the components of Hadoop 1 HDFS is responsible for data storage, and MapReduce, which sits above HDFS, manages resources and performs data processing; as a result of this, the system performance is heavily impacted.	The Hadoop Distributed File System, or HDFS, is utilised once again for storage in Hadoop 2, while YARN, a Resource Management system, is layered over HDFS to perform its duties. In essence, it distributes the available resources and ensures that everything continues to function normally. YARN actually shares the load of MapReduce. Hence the system performance improves
	 The diagram for HADOOP 1 shows a vertical stack. At the top is a box labeled 'MapReduce (Resource Management + Data Processing)'. Below it is a box labeled 'HDFS'.	 The diagram for HADOOP 2 shows a vertical stack. At the top is a box labeled 'MapReduce + Other Types of Jobs'. Below it is a box labeled 'YARN (Resource Management)'. At the bottom is a box labeled 'HDFS'.
Limitation	The architecture of Hadoop 1 is known as Master-Slave architecture. One master rules over a large number of slaves in this arrangement. In the event that the master node experienced a catastrophic failure, the cluster would be wiped out regardless of the quality of the slave nodes. Again, in order to create that cluster, you have to copy system files, picture files, and so on onto another machine, which takes an excessive amount of time and is something that enterprises just cannot accept.	Hadoop 2 utilises the same Master-Slave architecture as its predecessor. However, this is made up of a number of different masters (also known as active namenodes and standby namenodes) and a number of different slaves. In the event that this master node suffers a crash, the standby master node will take control of the network. You are able to create a wide variety of different active-standby node combinations. As a result, the issue of having a single point of failure will be resolved with Hadoop 2.

EcoSystem	Oozie is a simplified version of a workflow scheduler. It is responsible for determining the specific times at which jobs will run based on the dependencies between them. Pig, Hive, and Mahout are examples of data processing tools that sit above Hadoop and execute their job there. Sqoop is an application that can import and export structured data. Using a SQL database, it is possible to immediately import and export data into HDFS. Flume is a tool that is used to import and export streaming data as well as unstructured data.	
Support	No Support for Microsoft Windows is available for Hadoop 1.x	Support of Microsoft Windows available for Hadoop 2.x

🔗Check Your Progress 7

- What is Hadoop? Discuss the components of Hadoop.

.....

.....

.....
- Discuss the role of HDFS and MapReduce in Hadoop Architecture

.....

.....

.....
- Discuss the relevance of various nodes in HDFS architecture

.....

.....

.....
- Explain how the master/slave process works in HDFS architecture.

.....

.....

.....

.....

5. What do you understand by the Map phase and reduce phase in MapReduce architecture?

.....

.....

.....

6. List and explain the various daemons involved in the functioning of MapReduce.

.....

.....

.....

7. Differentiate between Apache Hadoop-1 and Hadoop-2.

.....

.....

.....

11.5 SUMMARY

This Unit provided a foundational understanding of NoSQL databases, detailing the various types such as column-based, graph-based, key-value, and document-oriented databases. It highlighted NoSQL's growing adoption across industries due to its ability to support mission-critical applications, reduce data dispersion, lower ownership costs, and mitigate risks—though selecting the right database platform remains a complex task.

Each NoSQL model has its strengths and limitations. Column-store databases, while powerful, may face consistency issues due to the way data is written to disk. Graph databases excel in delivering intuitive, high-performance content structures, particularly useful in applications like social networks. Key-value stores offer simplicity and speed but may fall short in handling complex operations such as financial transactions. Document databases are widely used for their flexibility and support for diverse data types, making them ideal for analytical applications. Combining relational and non-relational systems can lead to more efficient data processing pipelines.

In addition, the Unit covered the key concepts of Big Data, including its defining characteristics—volume, velocity, variety, and veracity—and its practical applications. It also explained the differences between structured, semi-structured, and unstructured data, and compared Big Data systems with traditional data warehouses. Important infrastructure components such as distributed file systems were discussed, culminating in a comparison of technologies like MapReduce, HDFS, and Apache Hadoop (versions 1 and 2 with YARN).

11.6 SOLUTIONS/ANSWERS

☞ Check Your Progress 1

1. What is NoSQL? NoSQL (Not Only SQL) is a type of database designed to store and manage large volumes of unstructured, semi-structured, or structured data. It offers flexible data models and is well-suited for distributed systems, real-time web apps, and big data use cases.
2. What are the features of NoSQL databases?
 - Schema-less or flexible data models
 - Horizontal scalability
 - High performance for read/write operations
 - Support for unstructured and semi-structured data
 - Designed for distributed and cloud-based systems
 - Includes various types: key-value, document, column, and graph databases
3. Differentiate between NoSQL and SQL:

Feature	SQL (Relational)	NoSQL (Non-relational)
Data Model	Table-based with fixed schema	Flexible schema (key-value, document, etc.)
Scalability	Vertical (scale-up)	Horizontal (scale-out)
Query Language	Structured Query Language (SQL)	Varies (e.g., JSON queries, APIs)
Data Integrity	Strong consistency (ACID)	Often eventual consistency (BASE)
Best For	Structured data and complex queries	Large-scale, unstructured, real-time data

☞ Check Your Progress 2

1. How Does a Column Database Work? A column database (or column-family database) stores data in columns instead of rows. Each column stores values of a single attribute for multiple records, which allows for faster querying and better compression—especially when dealing with large datasets where only certain attributes are needed. It is ideal for analytical workloads and big data applications. Examples include Apache Cassandra and HBase.
2. What are the Different Graph Database Examples? Graph databases store data in nodes (entities) and edges (relationships), making them suitable for relationship-heavy data. Popular graph database examples include:
 - Neo4j

- Amazon Neptune
- ArangoDB
- OrientDB
- Microsoft Azure Cosmos DB (with Gremlin API)

3. Explain Document-Based NoSQL Database: A document-based NoSQL database stores data as documents, typically in JSON, BSON, or XML format. Each document is self-contained and can have a flexible schema, making it suitable for storing complex, hierarchical data. These databases are ideal for content management systems, catalogs, and real-time analytics. Examples include MongoDB and CouchDB.

☞ Check Your Progress 3

1. What does "Big Data" mean? Big Data refers to extremely large and complex data sets generated from various sources at high speed. These datasets are too vast to be processed using traditional data management tools and require advanced methods for storage, processing, and analysis to extract meaningful insights.
2. List the components required by a Big Data system to function properly:
 - Data Sources Layer: Collects data from various inputs (e.g., sensors, web logs, social media, IoT devices).
 - Management and Storage Layer: Stores and manages the incoming data in a suitable format.
 - Analysis Layer: Processes and analyzes the stored data to derive insights.
 - Consumption Layer: Delivers the analysis results to users or systems.
 - Data Ingestion Tools: For collecting and importing data (e.g., Apache Flume, Kafka).
 - Distributed File Systems: For storing large volumes of data (e.g., HDFS).
 - Processing Frameworks: For computation and analytics (e.g., MapReduce, Spark).
 - Data Governance and Security Tools: To ensure data quality, privacy, and compliance.
3. Discuss the essential characteristics of Big Data: Big Data is commonly defined by the 4 Vs:
 - Volume: The vast amount of data generated every second.
 - Velocity: The speed at which data is produced and needs to be processed.
 - Variety: The different types and formats of data (structured, semi-structured, unstructured).
 - Veracity: The accuracy, reliability, and trustworthiness of the data.

☞ Check Your Progress 4

1. Five Applications of Big Data:

- **Healthcare:** Predictive analytics for disease detection and patient care.
- **Retail:** Personalized recommendations and customer behavior analysis.
- **Finance:** Fraud detection and algorithmic trading.
- **Manufacturing:** Predictive maintenance and supply chain optimization.
- **Social Media:** Sentiment analysis and trend prediction.

2. Comparison of Structured, Semi-Structured, and Unstructured Data:

Feature	Structured Data	Semi-Structured Data	Unstructured Data
Format	Fixed schema (tables, rows, columns)	Flexible schema (tags, keys)	No predefined format
Examples	SQL databases, spreadsheets	XML, JSON, NoSQL databases	Emails, images, videos, social media posts
Storage	Relational databases	NoSQL databases, XML/JSON files	Data lakes, file systems
Ease of Analysis	Easy	Moderate	Difficult

3. Role of Data Types in Decision-Making:

- **Structured Data:** Provides quantifiable insights for quick, accurate decisions (e.g., sales trends).
- **Semi-Structured Data:** Adds context and flexibility, enabling better understanding of diverse sources (e.g., customer reviews).
- **Unstructured Data:** Offers deep insights through content analysis, especially in areas like sentiment, image recognition, and behavior patterns.

☞ Check Your Progress 5

1. Similarities between Big Data and Data Warehouse:

- **Data Storage:** Both store large volumes of data.
- **Data Analysis:** Used for analyzing data to support decision-making.
- **Data Integration:** Combine data from multiple sources.
- **Historical Data:** Maintain historical records for trend analysis.
- **Business Intelligence:** Support reporting and insights generation.

2. Comparison of Big Data and Data Warehouse:

Feature	Big Data	Data Warehouse
Data Type	Structured, semi-structured, unstructured	Mainly structured
Volume & Variety	Very large, diverse formats	Moderate to large, structured format
Processing	Real-time or batch (via tools like Hadoop, Spark)	Batch processing (ETL processes)
Scalability	Highly scalable (distributed systems)	Limited scalability (traditional systems)
Use Case	Advanced analytics, machine learning	Reporting, OLAP, business intelligence

🔍 Check Your Progress 6

- Describe the term "Distributed File System" in the context of Big Data:
A Distributed File System (DFS) is a storage system that allows data to be stored across multiple machines or nodes, enabling scalable and fault-tolerant access to large datasets, commonly used in Big Data platforms like Hadoop.
- What is the primary objective of the Distributed File System?
The primary objective is to store and manage large volumes of data across multiple systems while ensuring high availability, fault tolerance, and efficient data access.
- Explain the components of the Distributed File System:
 - NameNode (or Metadata Server): Manages metadata and file directory structure.
 - DataNodes (or Storage Nodes): Store actual data blocks.
 - Client: Accesses and interacts with the DFS for read/write operations.
- Features of Distributed File System:
 - Scalability: Can handle large data volumes.
 - Fault Tolerance: Automatically recovers from node failures.
 - High Availability: Ensures continuous access to data.
 - Data Replication: Copies data across nodes for safety.
 - Transparency: Hides complexity from users.
- Ways to Implement Distributed File System in Practice:
 - HDFS (Hadoop Distributed File System)
 - Google File System (GFS)
 - Amazon S3 (for cloud-based DFS-like storage)
 - Ceph and GlusterFS (open-source DFS solutions)

- Microsoft DFS (for enterprise file sharing)

6. Advantages and Disadvantages of Distributed File Systems:

Advantages:

- High fault tolerance
- Scalability
- Efficient data access
- Data redundancy
- Cost-effective for large data

Disadvantages:

- Complex setup and maintenance
- Network dependency
- Latency issues during access
- Security concerns across nodes
- Data consistency challenges

🔍 **Check Your Progress 7**

1. What is Hadoop? Discuss the components of Hadoop: Hadoop is an open-source framework for processing and storing large-scale data across distributed systems. Main components:
 - HDFS (Hadoop Distributed File System): For data storage.
 - MapReduce: For data processing.
 - YARN (Yet Another Resource Negotiator): Manages resources and job scheduling.
 - Hadoop Common: Utilities and libraries used by other Hadoop modules.
2. Role of HDFS and MapReduce in Hadoop Architecture:
 - HDFS: Provides scalable and fault-tolerant storage by distributing data across nodes.
 - MapReduce: Processes large data sets in parallel by dividing tasks into map and reduce phases.
3. Relevance of Various Nodes in HDFS Architecture:
 - NameNode (Master): Manages metadata and file directory structure.
 - DataNode (Slave): Stores actual data blocks and handles read/write requests.
 - Secondary NameNode: Periodically backs up NameNode metadata (not a standby node).
4. Master/Slave Process in HDFS Architecture:
 - The NameNode (master) maintains metadata, while multiple DataNodes (slaves) store actual data.
 - Clients request file operations from the NameNode, which directs

them to appropriate DataNodes.

5. Map Phase and Reduce Phase in MapReduce Architecture:

- Map Phase: Splits input data into key-value pairs and processes them in parallel.
- Reduce Phase: Aggregates and summarizes the intermediate key-value pairs to produce the final output.

6. Daemons in MapReduce Functioning:

- JobTracker (Hadoop 1): Manages jobs and resource scheduling.
- TaskTracker (Hadoop 1): Executes map and reduce tasks on nodes.
- ResourceManager (Hadoop 2/YARN): Manages cluster resources.
- NodeManager (Hadoop 2): Manages resources and monitors tasks on individual nodes.
- ApplicationMaster (Hadoop 2): Manages execution of a single job.

7. Difference Between Hadoop-1 and Hadoop-2:

Feature	Hadoop-1	Hadoop-2
Resource Management	JobTracker	YARN (Resource Manager + Application Master)
Scalability	Limited	Highly scalable
Fault Tolerance	Single JobTracker (bottleneck)	Better fault tolerance with YARN
Data Processing	Only MapReduce	Supports other models (Spark, Tez, etc.)
Cluster Utilization	Less efficient	Improved resource utilization

11.7 FURTHER READINGS

- 1) Next Generation Databases: NoSQL and Big Data 1st ed. Edition, G. Harrison, Apress, December 26, 2015.
- 2) Shashank Tiwari, Professional NoSQL, 1st Edition, Wrox, September 2011.
- 3) <https://www.kdnuggets.com/>
- 4) C. Lam, "Introducing Hadoop", in Hadoop in Action, MANNING, 2011.
- 5) D. Borthakur, "The Hadoop distributed file system: architecture and design," Hadoop Project Website [online]. Available: <http://hadoop.apache.org/core/docs/current/hdfs design.pdf>